

Koopman Operator Approach Data-Driven Optimal Control Algorithm for Autonomous Vehicles with various characteristics

Hakjoo Kim, Hwan-Hong Lee, Seok-Cheol Kee, *Member, IEEE*

Abstract— The complex mathematical model of autonomous vehicles makes it difficult for system identification due to a combination of non-linearity and uncertainty. Various strategies have been proposed to address the difficulty in system identification, as it significantly influences the precise path-tracking performance of autonomous vehicles. This paper proposes a Koopman operator approach data-driven optimal control algorithm for path-tracking of autonomous vehicles. To identify mathematical model's various vehicle types of autonomous vehicle driving data were acquired in virtual simulation and real-world environments. An integrated linear model was identified using the Koopman operator neural network and the acquired driving data of autonomous vehicles. The identified integrated linear model was incorporated into a model predictive control algorithm designed for the path-tracking of autonomous vehicles. Reasonable path tracking performance was confirmed through performance evaluations conducted in path-tracking scenarios using various vehicle types for real and virtual vehicles in the real autonomous driving proving ground C-track and CARLA simulator environments.

Index Terms—Koopman operator, deep neural network, model predictive control, autonomous vehicles, path-tracking

I. INTRODUCTION

Autonomous driving technology continues to attract attention, and demand from various school and industries is also increasing. As demand for autonomous driving technology increases, open source-based autonomous driving software platforms are being developed to overcome the limitations of workforce, time, and technology required for research and development. Autonomous driving open-source platforms such as Baidu's Apollo [1], comma.ai's openpilot [2], and TIER IV's Autoware [3] have been released. Among them, Autoware has developed based on ROS (Robot Operating Systems) [4] and is managed by the Autoware Foundation along with the TIER IV. Autonomous driving technology includes localization, perception, decision-making, path planning, and control algorithms. Autoware supports a full-stack open-source platform for the mentioned autonomous driving technologies. Deriving the mathematical model of autonomous vehicles that represents the physical characteristics of the system significantly influences control performance. An autonomous vehicle's Mathematical models affects the driving characteristics and control performance. Therefore, it is essential to design a model for autonomous vehicles that integrates various systems. The mathematical model varies based on the vehicle type, leading to different

driving characteristics in autonomous vehicles. Therefore, designing an integrated system model for autonomous vehicles incorporating various vehicle types and systems is essential.

The Koopman operator-based linear system identification and modeling method has received attention for controlling nonlinear systems such as manipulators and quadrotors, including hydrodynamic systems [5-7]. The Koopman operator describes a measurement function of the system state, an infinite-dimensional linear transformation acting on the Hilbert space of the lifted observation space [8], [9]. As an operator that can linearize the characteristics of a nonlinear system, the Koopman operator was approximated through dimensionality reduction methods such as dynamic mode decomposition (DMD) and extended dynamic mode decomposition (EDMD) [10], [11]. Additionally, a data-based linear model approximation method using deep learning was also conducted [12-14]. Along with the nonlinear systems, studies have been conducted using Koopman operator-based linearization modeling and control algorithms for autonomous vehicles.

Yu et al. [15] derived a bilinear Koopman operator-based vehicle model to approach the mismatch between the nonlinear system of autonomous vehicles and data-driven linear models. The derived model was reflected in model predictive control (MPC) for real-time route planning. Feng et al. [16] proposed a hierarchical control strategy for vehicle platoons that considered the longitudinal and lateral coupling characteristics of the vehicle by linearizing the vehicle's three degrees of freedom dynamic model with the Koopman operator. Švec et al. [17] proposed a predictive control algorithm for direct yaw moment control (DYC) of vehicles using the Koopman operator, Enhanced EDMD, and finite-dimensional numerical approximation. Kim et al. [18] proposed a data-driven modeling control methodology using the Koopman operator for the lane-keeping system of autonomous vehicles. Kim et al. [19] the same author as [18], reflected stochastics in MPC to compensate for the uncertainty of the model approximated by the Koopman operator and EDMD. It was confirmed that the algorithm proposed by Kim ensures reasonable path-following performance for autonomous vehicles. Gupta et al. [20] proposed MPC using a linearized dynamics model based on the Koopman operator for eco-friendly driving of autonomous vehicles. Buzhardt [21] et al. proposed a data-driven control algorithm for intelligent suspension, motion control, and path following in which the Koopman operator was applied to

*This work was supported by Innovative Human Resource Development for Local Intellectualization program through the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT)(IITP-2024-2020-0-01462). This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2022R1A5A8026986).

H. Kim, and H.H. Lee are with the Department of Smart Car Engineering, Chungbuk National University, Cheongju 28644, South Korea, {hakjoo0926, hwanhong57} @ chungbuk.ac.kr.

S.C. Kee is with the Department of Intelligent Systems and Robotics, Chungbuk National University, Cheongju 28644, South Korea, sckee@ chungbuk.ac.kr.

obtain a linear expression for the vehicle-tire-terrain interaction dynamics and control input effects of an off-road vehicle. Folkestad [22] and others designed a nonlinear model predictive control algorithm by expressing control-affine dynamics as a lift bilinear model using the Koopman standard transformation to compensate for the nonlinear operating effects inherent in the system. Cibulka et al. [23] presented results of vehicle dynamic model identification using the Koopman operator and EDMD. Wang et al. [24] integrated the nonlinear modeling function of a neural network and Koopman-based MPC and applied the model mismatch between the trained Koopman linear model and nonlinear dynamics to the path-tracking problem of an autonomous vehicle using an online and offline constraint-tightening approach. Wang [25] et al. proposed a model predictive controller that uses the Koopman deep neural network to acquire data from various simulation scenarios and identify the system of vehicle dynamics. Xiao et al. [26] proposed deep learning-based EDMD (Deep-EDMD) to identify vehicle dynamics models, and the proposed system identifier was reflected in MPC and applied to track the speed profile of autonomous vehicles. Xiao [27], the same author as [26], proposed a linear MPC using deep direct Koopman, a deep learning approach for longitudinal and lateral integrated control of autonomous vehicles.

In this study, we propose a Koopman operator-based data-driven optimal control algorithm for achieving path-tracking performance through the identification of the nonlinear kinematic vehicle system of autonomous vehicles. Driving data from various vehicle types, including virtual and real vehicles, were acquired, and system identification was performed using deep Koopman neural networks (DKNN) with an Autoencoder structure. The identified vehicle models were integrated into model predictive control (MPC), and performance evaluations were conducted. The contributions presented in this paper are as follows:

- The acquisition of driving data in virtual simulation and real-world environments with various types of autonomous vehicles.
- An integrated linear model for autonomous vehicles was derived through DKNN, and the derived linear model was integrated into the MPC algorithm to confirm that relatively reasonable path-tracking performance was compensated by relatively small lateral errors and reduced chattering phenomenon compared to Autoware Kinematic MPC(K-MPC).
- By using real vehicles and various types of virtual vehicles provided by the CARLA simulator [28], we compensated for the reasonable path-tracking performance of the integrated linear model with MPC.

The structure of this paper, including the main content, is outlined as follows. Section II presents the Koopman operator-based data-driven optimal control algorithm. Section III describes the performance evaluation in simulation and real-world scenarios. Finally, the paper concludes with a summary, analysis of limitations, and suggestions for future research.

II. DATA-DRIVEN OPTIMAL CONTROL ALGORITHM USING KOOPMAN OPERATOR

A. Koopman Operator with Deep neural network

In this chapter, we present a data-driven approach to optimal control algorithms for linearizing the nonlinear model of autonomous vehicles. Fig. 1 illustrates the nonlinear kinematic model representing the lateral motion of an autonomous vehicle. In Fig. 1, v_x , θ , β , and δ_f represent the longitudinal velocity, yaw angle, vehicle sideslip angle, and front wheel steering angle. δ_{des} represents the target wheel steering angle. L is the wheelbase, R_t is the turning radius expressed as the reciprocal of the curvature ρ .

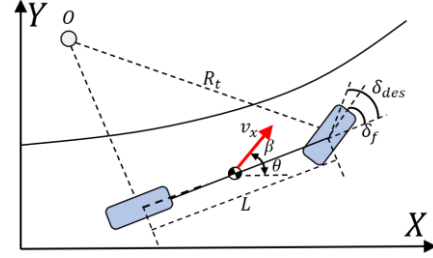


Figure 1. A nonlinear kinematics model of front wheel steering vehicle

The equations for the time-varying nonlinear system of the vehicle can be defined using states $x = [v_y, \theta, \delta_f]$, and control input $u = \delta_{des}$ as follows [29].

$$x_{k+1} = f(x_k, u_k) \quad (1)$$

Where k represents discrete time. The function f denotes the dynamical mapping, expressed in the Euclidean space as $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$. The observable extended states η can be expressed as follows.

$$\eta_k = \begin{bmatrix} x_k \\ u_k \end{bmatrix} \quad (2)$$

Where $\eta \in \mathbb{R}^{n+m}$. The discretized model of the extended states η can be expressed as follows, along with the observable measurement functions F .

$$\eta_{k+1} = F(\eta_k) = \begin{bmatrix} f(x_k, u_k(0)) \\ \sigma u_k \end{bmatrix} \quad (3)$$

In equation (3), σ represents the left shift operator. It can be expressed as $\sigma u_k = u_{k+1}$. The Koopman operator K can be represented as follows, along with the extended states η , the measurements function F and the lifting function ψ .

$$K(\eta_k) = \psi(F(\eta_k)) = \psi(\eta_{k+1}) \quad (4)$$

Where $K: \mathcal{F} \rightarrow \mathcal{F}$ represents the Koopman operator. $\mathcal{F}$ is the extended function space belonging to $\mathbb{R}$. The measurement states or output states y_k is defined using the output matrix $C \in \mathbb{R}^{m \times n}$ and the lifted states z_k . The system matrix $A \in \mathbb{R}^{n \times n}$, and the input matrix $B \in \mathbb{R}^{n \times l}$.

$$z_{k+1} = [A \ B] \begin{bmatrix} z_k \\ u_k \end{bmatrix} = Az_k + Bu_k \quad (5)$$

$$y_k = Cz_k \quad (6)$$

DKNN trains the dynamic properties of linear systems and captures the nonlinear behavior of the system. This allows the model to effectively represent the original high-dimensional dynamical system while maintaining the low-dimensional state representation. Fig. 2 shows a diagram of DKNN for nonlinear car model identification. DKNN was developed in the Python 3.X environment and implemented using the PyTorch library. This architecture compresses input data and extracts features using the Autoencoder structure.

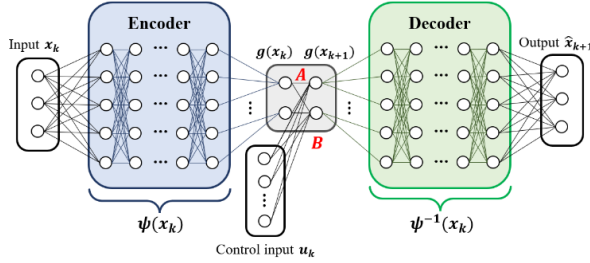


Figure 2. Overall diagram of deep Koopman neural network

The encoder network converts observable states x_k into a lower-dimensional mapping function $g(x_k)$. The encoder has a multi-layer perceptron structure, and each layer consists of a linear transform and a ReLU activation function. A linear propagation layer calculates the $g(x_{k+1})$ using the output of the encoder network and control input u_k . The linear propagation layer can be expressed as follows.

$$g(x_{k+1}) = W_p(g(x_k) + u_k) \quad (7)$$

Where, W_p represent the weight matrix of the linear propagation layer. The decoder network serves the role of reconstructing $g(x_{k+1})$ to the original states x_{k+1} . It has a multi-layer perceptron architecture identical to the encoder, with linear transformations and ReLU activation functions applied. The decoder is responsible for mapping the states reduced to a lower dimension back to the original dimensions. The matrix A , and B of the linear system mentioned in equation (5) are approximated by accessing the weight matrix W_p of the neural network model.

$$A = \begin{bmatrix} W_{p(1,1)} & W_{p(1,2)} & \dots & W_{p(1,n)} \\ W_{p(2,1)} & W_{p(2,2)} & \dots & W_{p(2,n)} \\ \vdots & \vdots & \ddots & \vdots \\ W_{p(n,1)} & W_{p(n,2)} & \dots & W_{p(n,n)} \end{bmatrix} \quad (8)$$

$$B = \begin{bmatrix} W_{p(1,n+1)} & W_{p(1,n+2)} & \dots & W_{p(1,n+l)} \\ W_{p(2,n+1)} & W_{p(2,n+2)} & \dots & W_{p(2,n+l)} \\ \vdots & \vdots & \ddots & \vdots \\ W_{p(n,n+1)} & W_{p(n,n+2)} & \dots & W_{p(n,n+l)} \end{bmatrix} \quad (9)$$

The overall loss function L_t is designed using the loss function of the Autoencoder L_a and the metric loss function L_m , and it is formulated as follows.

$$L_a = \sum_{i=1}^p (x_i - \hat{x}_i)^2 \quad (10)$$

$$L_m = \left\| \sum_{i=1}^n g(x_{i+1}) - g(x_i) - \sum_{i=1}^p \|x_{i+1} - x_i\| \right\| \quad (11)$$

$$L_t = L_a + \gamma_l L_m \quad (12)$$

Where, γ_l represents the weights L_m in L_t loss. In training, the neural network determines its weights to minimize the overall loss function [30], [31]. This optimization is achieved through the utilization of the Adam solver, an algorithm based on gradient descent.

B. Model predictive control algorithm

MPC calculate optimal control inputs to use linear and nonlinear system models to determine a finite number of prediction steps. And it is designed to solve control problems by considering the physical constraints of the system and integrating a prediction process for future states in each control period. The linear system in equation (5), extended from the initial states x_0 and the initial control input u_0 to prediction steps N_p , can be derived as a prediction model.

$$Y = \zeta x_0 + GU \quad (13)$$

$$Y = [Cx_1 \ Cx_2 \ Cx_3 \ \dots \ Cx_{N_p}]^T \quad (14)$$

$$\zeta = [CA \ CA^2 \ CA^3 \ \dots \ CA^{N_p}]^T \quad (15)$$

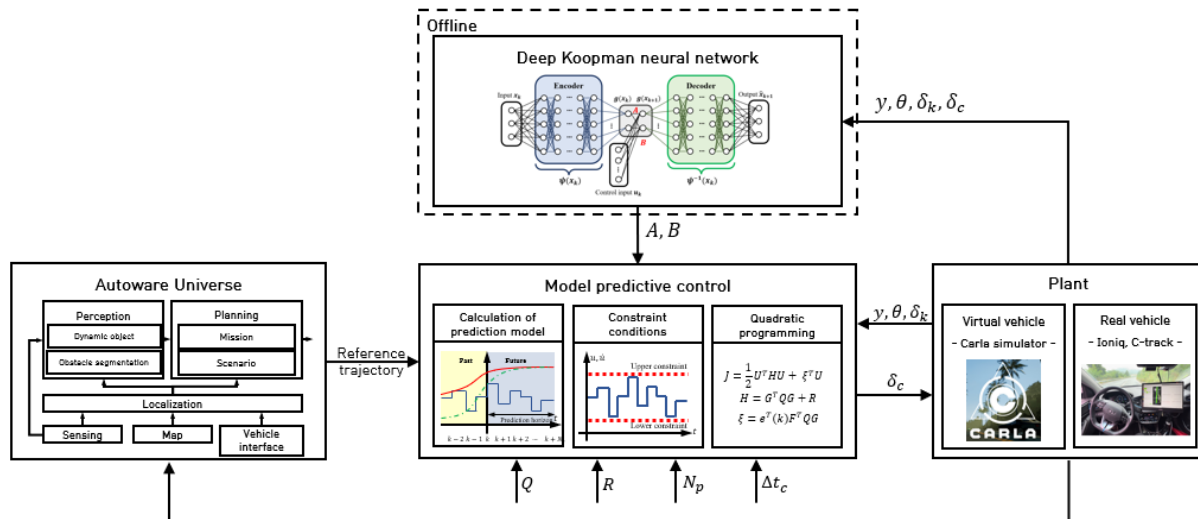


Figure 3. Model schematics of autonomous vehicles architecture with deep Koopman operator with model predictive control algorithm

$$G = \begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA^{Np-1}B & CA^{Np-2}B & CA^{Np-3}B & \dots & CB \end{bmatrix} \quad (16)$$

$$U = [u_0 \ u_1 \ u_2 \ \dots \ u_{Np-1}]^T \quad (17)$$

The cost function of the MPC is designed using the prediction model in equation (13), the reference trajectory Y_{ref} , steady state input U_{ref} , and weight matrices Q and R , and it is formulated as follows.

$$J = (Y - Y_{ref})^T Q (Y - Y_{ref}) + (U - U_{ref})^T R (U - U_{ref}) \quad (18)$$

Where U_{ref} can be calculated using L and road curvature ρ as $U_{ref} = \tan^{-1}(L\rho)$. The Q and R are positive semi-definite matrices. When substituting equation (13) into equation (18), it can be derived into the following quadratic form.

$$J = \frac{1}{2} U^T H U + \zeta^T U \quad (19)$$

$$H = G^T Q G + R \quad (20)$$

$$\zeta = (\zeta x_0)^T Q G - U_{ref}^T R \quad (21)$$

A quadratic programming solver was applied to reflect the physical constraints of the system and derive optimal control inputs. Boundary constraints condition and inequality constraints condition were used to apply constraints to control input. Equations (22) and (23) describe the constraints as follows.

$$[-u_{lim} \ -u_{lim} \ \dots \ -u_{lim}]^T \leq U \leq [u_{lim} \ u_{lim} \ \dots \ u_{lim}]^T \quad (22)$$

$$\begin{bmatrix} -1 & 1 & 0 & \dots & \dots & 0 \\ 1 & -1 & 0 & \dots & \dots & 0 \\ 0 & -1 & 1 & \dots & \dots & 0 \\ 0 & 1 & -1 & \dots & \dots & 0 \\ 0 & 0 & 0 & \ddots & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & -1 & 1 \\ 0 & 0 & 0 & \dots & 1 & -1 \end{bmatrix} U \leq \begin{bmatrix} \dot{u}_{lim} \Delta t_c \\ -\dot{u}_{lim} \Delta t_c \\ \dot{u}_{lim} \Delta t_c \\ -\dot{u}_{lim} \Delta t_c \\ \dot{u}_{lim} \Delta t_c \\ \vdots \\ -\dot{u}_{lim} \Delta t_c \end{bmatrix} \quad (23)$$

Where u_{lim} represents the limits of the control input. $\dot{u}_{lim}$ and Δt_c respectively denote the limits of the derivative of control input and the control period of the MPC, with an applied value of approximately 33 Hz.

III. PERFORMANCE EVALUATION IN VIRTUAL AND REAL VEHICLES

This section describes the driving data acquisition from the virtual vehicles with the real vehicles, and performance evaluation. The virtual and real vehicles were configured using the Autoware Universe autonomous driving open-source platform. Fig. 3 illustrates the overall model schematics of the proposed algorithms, including data acquisition, Koopman operator-based vehicle linear model classification, and MPC.

A. Driving Data Acquisition in Virtual and Real Vehicles

For driving data acquisition in a simulation environment with virtual vehicles, the CARLA simulator was employed. Additionally, real-world autonomous vehicle driving data was collected on the autonomous driving testbed, C-track [32], at Chungbuk National University. The C-track circuit test area is roughly 1.4km in length, consisting of a two-lane motorway which includes a BRT bus lane and bus stops. In the simulator environment, sedan, small, truck, and SUV models were applied, and in the real world the real vehicle IONIQ HEV was used. Fig. 4 presents the driving datasets of autonomous vehicles obtained from both real vehicles and the CARLA simulator. Case-1 features the sedan model, Case-2 presents the SUV model, and Case-3 involves the application of the real vehicle. where the longitudinal velocity was applied from 0 to 20 km/h.

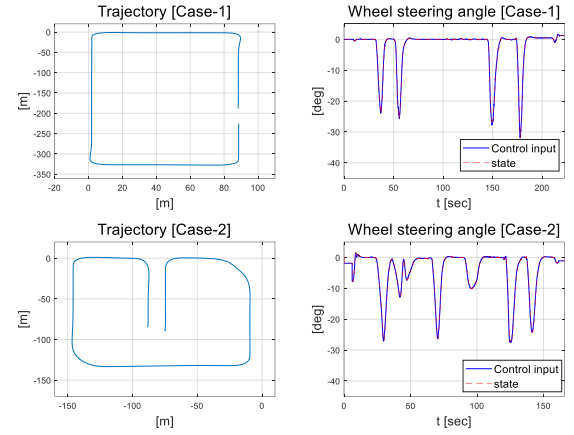


Figure 4. An autonomous vehicle driving data in virtual environments

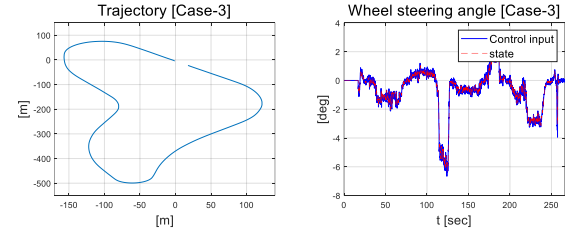


Figure 5. An autonomous vehicle driving data in real-world (C-track)

B. Results of Evaluation in Real Vehicle

In this subsection, we present the results of performance evaluation in the real-world. The comparison between K-MPC and MPC integrated with DKNN (DKNN-MPC) was conducted using the real vehicle on the C-track testbed. Table 1 provides the MPC representative control gains that were adopted. The K-MPC and DKNN-MPC were determined with the same control gain values. Fig. 6 shows the autonomous vehicle trajectory of autonomous vehicles with real vehicles. Fig. 7 displays the wheel angle and control input for K-MPC and DKNN-MPC. As can be seen in Fig. 7, when DKNN-MPC was applied, it was confirmed that the chattering phenomenon was relatively reduced compared to K-MPC. Fig. 8 represents the lateral position error and yaw error. It can be confirmed that, in the case of DKNN-MPC, the relative lateral error was smaller than that of K-MPC, but the yaw error had a relatively large value. Fig. 9, representing the error histogram,

and Table 2, presenting mean, standard deviation (STD), and root mean squared (RMS) values in real vehicles.

TABLE I. CONTROL PARAMETER FOR PERFORMANCE EVALUATION

Control gain	Description	Value	Unit
N_p	Prediction step	100	-
Δt_c	Control period	0.001	-
Q	Weight matrix	$\begin{bmatrix} 0.001 & 0 \\ 0 & 0 \end{bmatrix}$	-
R	Weight matrix	1.0	-
τ_c	Constant time delay	0.001	-
u_{lim}	Limitation of control input	1	rad

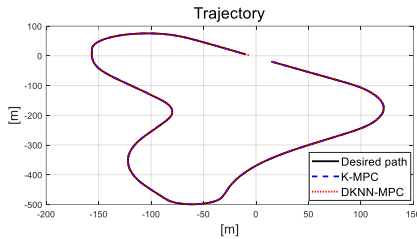


Figure 6. Results of vehicle trajectory in real vehicle compared K-MPC and DKNN-MPC

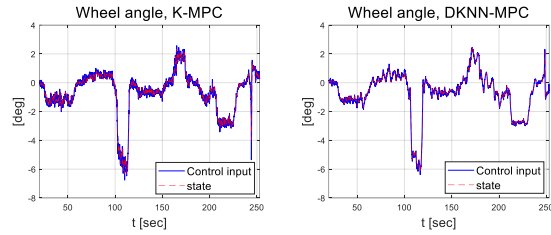


Figure 7. Results of control input and wheel angle in real vehicle compared K-MPC and DKNN-MPC

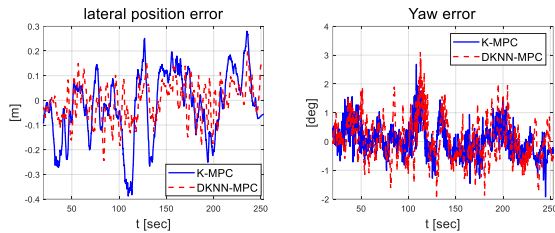


Figure 8. Results of lateral position error and yaw error in real vehicle compared K-MPC and DKNN-MPC

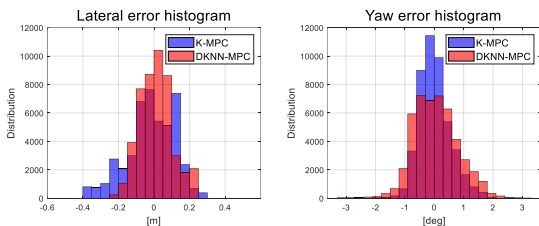


Figure 9. Results of histogram of lateral position error and yaw error in real vehicle

TABLE II. ERROR DISTRIBUTION IN REAL VEHICLE

States	Division	Mean	STD	RMS
Lateral position error, [m]	K-MPC	-0.0170	0.1334	0.1345
	DKNN-MPC	0.0113	0.0902	0.0909
Yaw error, [deg]	K-MPC	0.0197	0.5335	0.5338
	DKNN-MPC	0.0542	0.7575	0.7594

In Fig. 9, the lateral position error histogram, DKNN-MPC has a more compact distribution about 0. In the yaw error histogram, K-MPC has a more compact distribution of about 0 compared to DKNN-MPC. These trends are like the quantitative indicators provided in Table 2 representing the error distribution.

Results of Evaluation in Virtual Vehicles with Various Vehicle Types

This subsection describes the performance evaluation results for various types of autonomous vehicles using the CARLA simulator. The control gain values in Table 1 were consistently applied to virtual vehicles. As with the real vehicle applied in Figs. 6 to 9, the performance of K-MPC and DKNN-MPC was compared. To confirm reasonable control performance at different vehicles, performance evaluations were conducted using various models provided by CARLA, including the small model, SUV model, and Truck model. Fig. 10 represents the trajectory of various virtual autonomous vehicles.

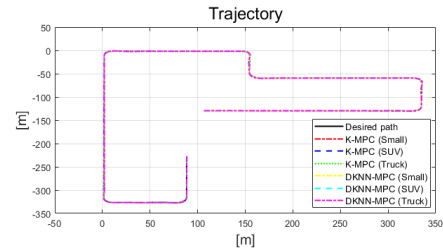


Figure 10. Results of vehicle trajectory in various types of virtual vehicles compared K-MPC and DKNN-MPC

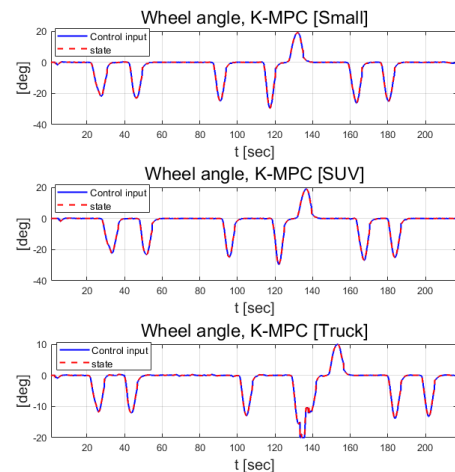


Figure 11. Results of control input and wheel angle in K-MPC with various types of virtual vehicles

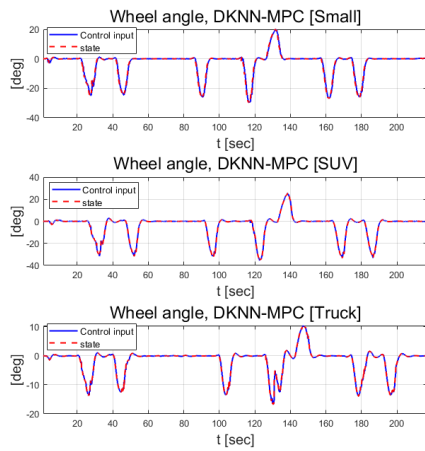


Figure 12. Results of control input and wheel angle in DKNN-MPC with various types of virtual vehicles

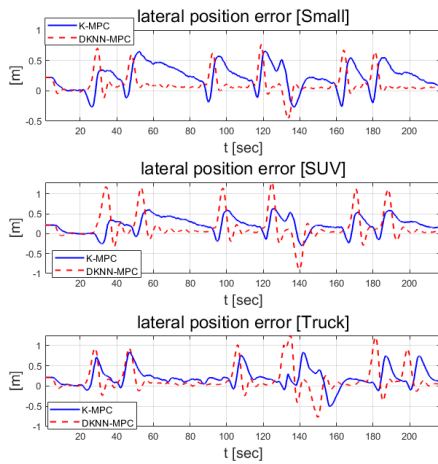


Figure 13. Results of of lateral position error in various types of virtual vehicles compared K-MPC and DKNN-MPC

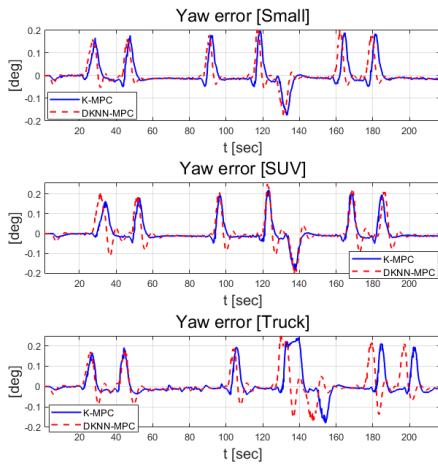


Figure 14. Results of of yaw error in various types of virtual vehicles compared K-MPC and DKNN-MPC

Fig. 11, and 12 show the wheel angle and control input for K-MPC and DKNN-MPC, respectively. Fig. 13 and 14 show the lateral position error and yaw angle error, respectively. In Fig. 13, when DKNN-MPC was applied, a relatively larger overshoot phenomenon occurred in the transient section than K-MPC, but it showed faster convergence.

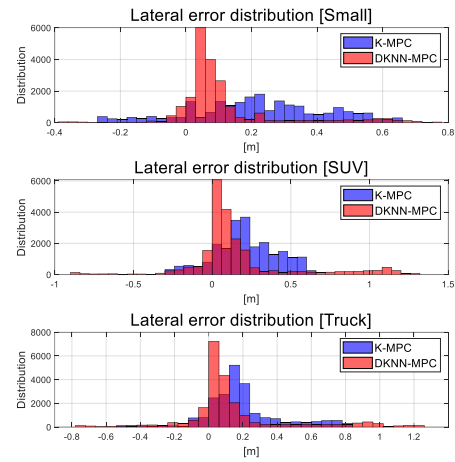


Figure 15. Results of histogram of lateral position error in various types of virtual vehicles

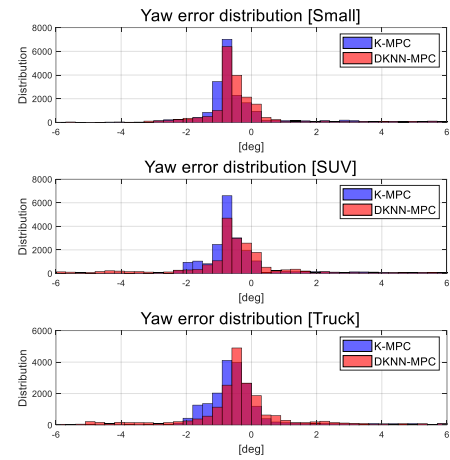


Figure 16. Results of histogram of yaw error in various types of virtual vehicles

TABLE III. ERROR DISTRIBUTION IN VIRTUAL VEHICLES

States	Control	Model	Mean	STD	RMS
Lateral position error, [m]	K-MPC	Small	0.2199	0.2041	0.3000
		SUV	0.2139	0.1983	0.2917
		Truck	0.1972	0.2226	0.2974
	DKNN-MPC	Small	0.1105	0.1685	0.2034
		SUV	0.1627	0.3287	0.3679
		Truck	0.1433	0.3101	0.3416
Yaw error, [deg]	K-MPC	Small	0.0859	2.9354	2.9366
		SUV	0.1519	3.1403	3.1439
		Truck	0.5859	3.7797	3.8248
	DKNN-MPC	Small	0.2087	2.9335	2.9404
		SUV	0.2645	3.5986	3.6073
		Truck	0.3725	3.7804	3.7986

In Fig. 14, when DKNN-MPC was applied, it was confirmed that a large relative oscillation occurred in the transient section. In Fig. 11 and 14, the graph of the Truck model has a phenomenon where the steering angle and yaw

error values remain fixed for 130 to 140 sec. This phenomenon is because it is difficult for K-MPC to achieve reasonable control performance, resulting in stop-and-go behavior. Fig. 15 and 16 present histograms for lateral position error and yaw angle error, respectively. Table 3 shows the distribution of lateral position error and yaw error. In Fig. 15, 16, the lateral position error histogram, DKNN-MPC has a more compact distribution about 0. These graphs represent similar trends to those shown in Fig. 9. Moreover, in the yaw error graphs for the small model and Truck model, it can be confirmed that DKNN-MPC is more concentrated around 0 compared to K-MPC. In Table 3, DKNN-MPC has relatively smaller mean values compared to K-MPC.

IV. CONCLUSION

In this paper, we proposed a Koopman Operator Approach Data-Driven Optimal Control Algorithm for Autonomous Vehicles with various characteristics. The complex mathematical model of autonomous vehicles makes it difficult for system identification due to a combination of non-linearity and uncertainty. To identify of linear model of various types of autonomous vehicles, driving data from virtual and real vehicles were acquired. The acquired driving data were incorporated into a DKNN with an Autoencoder architecture, resulting in an integrated linear model. The derived linear model was then integrated into an MPC. The performance evaluations were conducted using the real vehicle IONIQ, and various types of virtual vehicles in real-world and CARLA simulator environments to confirm reasonable control performance. The results of the performance evaluation indicated that reasonable control performance was achieved. However, it was confirmed that the determination of MPC control gain values, beyond system identification, significantly influences control performance. To overcome these limitations, we plan to propose adaptive rules for real-time control gain determination in future research. The proposed control algorithm is expected to be applicable not only to autonomous mobile robots and vehicles, but also to various industrial fields.

ACKNOWLEDGMENT

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2022R1A5A8026986) and This work was supported by Innovative Human Resource Development for Local Intellectualization program through the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT)(IITP-2024-2020-0-01462)

REFERENCES

- [1] *Apollo* [Online]. Available: <https://github.com/ApolloAuto/apollo>
- [2] *commaai*. [Online]. Available: <https://github.com/commaai/openpilot>
- [3] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," *IEEE Micro*, vol. 35, no. 6, pp. 60–68, Nov./Dec. 2015.
- [4] Documentation—ROS Wiki. Accessed: Nov. 11, 2022. [Online]. Available: <http://wiki.ros.org/>
- [5] D. Bruder, X. Fu, R. B. Gillespie, C. D. Remy, and R. Vasudevan, "Koopman-based control of a soft continuum manipulator under variable loading conditions," *IEEE Trans. Robot.*, vol. 6, no. 4, pp. 6852–6859, Jul. 2021.
- [6] H. Wang, W. Liang, B. Liang, H. Ren, Z. Du, and Y. Wu, "Robust position control of a continuum manipulator based on selective approach and Koopman operator," *IEEE Trans. Ind. Electron.*, vol. 70, no. 12, pp. 12522–12532, Jan. 2023.
- [7] V. Zinged, and E. Bakolas, "Koopman operator based modeling for quadrotor control on SE (3)," *IEEE Control Syst. Lett.*, vol. 6, pp. 752–757, Jun. 2021.
- [8] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Data-driven approximations of dynamical systems operators for control," in *The Koopman Operator in Systems and Control*. New York, NY, USA: Springer, 2020, pp. 197–234.
- [9] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Nat. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, May 1931.
- [10] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Dynamic mode decomposition with control," *SIAM J. Appl. Dynamical Syst.*, vol. 15, no. 1, pp. 142–161, 2016.
- [11] M. Švec, Š. Ileš, and J. Matusko, "Model predictive control of vehicle dynamics based on the Koopman operator with extended dynamic mode decomposition," in *Proc. 22nd IEEE Int. Conf. Ind. Technol.*, 2021, vol. 1, pp. 68–73.
- [12] H. Shi, and M. Q. H. Meng, "Deep Koopman operator with control for nonlinear systems," *IEEE Trans. Robot.*, vol. 7, no. 3, pp. 7700–7707, Jun. 2022.
- [13] M. Bakhtiaridoust, F. N. Irani, M. Yadegar, and N. Meskin, "Data-driven sensor fault detection and isolation of nonlinear systems: Deep neural-network Koopman operator," *IET Control Theory & Applications*, vol. 17, no. 2, pp. 123–132, 2023.
- [14] X. Wang, Y. Kang, and Y. Cao, "Deep Koopman operator based model predictive control for nonlinear robotics systems," in *Proc. Int. Conf. Adv. Robot. Mechatronics*, 2021, pp. 931–936.
- [15] S. Yu, C. Shen, and T. Ersal, "Autonomous driving using linear model predictive control with a Koopman operator based bilinear vehicle model," *IFAC-PapersOnLine*, vol. 55, no. 24, pp. 254–259, 2022.
- [16] Y. Feng, S. Yu, E. Sheng, Y. Li, S. Shi, J. Yu, and H. Chen, "Distributed MPC of Vehicle Platoons Considering Longitudinal and Lateral Coupling," *IEEE Trans. Int. Trans. Syst.*, early access, pp. 1–18, Oct. 2023.
- [17] M. Švec, Š. Ileš, and J. Matusko, "Predictive Direct Yaw Moment Control Based on the Koopman Operator," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 6, pp. 2912–2919, May. 2023.
- [18] J. S. Kim, Y. S. Quan, and C. C. Chung, "Data-driven modeling and control for lane keeping system of automated driving vehicles: Koopman operator approach," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, pp. 1049–1055, 2022.
- [19] J. S. Kim, Y. S. Quan, and C. C. Chung, "K-SMPC: Koopman Operator-Based Stochastic Model Predictive Control for Enhanced Lateral Control of Autonomous Vehicles," 2023, *arXiv preprint arXiv:2310.10214*.
- [20] S. Gupta, D. Shen, D. Karbowski, and A. Rousseau, "Koopman model predictive control for eco-driving of automated vehicles," in *Proc. Amer. Control Conf. (ACC)*, Jun. 2022, pp. 2443–2448.
- [21] J. Buzhardt and P. Tallapragada, "A Koopman operator approach for the vertical stabilization of an off-road vehicle," *IFAC-PapersOnLine*, vol. 55, no. 37, pp. 675–680, 2022.
- [22] C. Folkestad and J. W. Burdick, "Koopman NMPC: Koopman-based learning and nonlinear model predictive control of control-affine systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 7350–7356.
- [23] V. Cibulka, T. Haniš, and M. Hromčík, "Data-driven identification of vehicle dynamics using Koopman operator," in *Proc. IEEE 22nd Int. Conf. Process Control*, 2019, pp. 167–172.
- [24] Y. Wang, Y. Yang, Y. Pu, and C. Manzie, "Data-driven predictive tracking control based on Koopman operators," 2022, *arXiv:2208.12000*.
- [25] R. Wang, Y. Han, and U. Vaidya, "Deep Koopman data-driven control framework for autonomous racing," in *Proc. Int. Conf. Robot. Autom.*

- (ICRA) Workshop Opportunities Challenges Auton. Racing, 2021, pp. 1–6.
- [26] Y. Xiao, X. Zhang, X. Xu, X. Liu, and J. Liu, “Deep neural networks with Koopman operators for modeling and control of autonomous vehicles,” *IEEE Trans. Intell. Veh.*, vol. 8, no. 1, pp. 135–146, Jun. 2022.
 - [27] Y. Xiao, X. Zhang, X. Xu, Y. Lu, and J. Lil, “DDK: A Deep Koopman Approach for Longitudinal and Lateral Control of Autonomous Ground Vehicles,” In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May. 2023, pp. 975–981.
 - [28] Carla [Online]. Available: <https://github.com/carla-simulator/carla>
 - [29] Tier IV, “MPC follower description- Autoware Documentation,” Mar. 2022. [Online]. Available: <https://web.archive.org/web/20220308221519/https://tier4.github.io>
 - [30] Y. Han, W. Hao, and U. Vaidya, “Deep learning of Koopman representation for control,” in *Proc. 59th IEEE Conf. Decis. Control (CDC)*, Dec. 2020, pp. 1890–1895.
 - [31] Y. Li, H. He, J. Wu, D. Katabi, and A. Torralba, “Learning compositional Koopman operators for model-based control,” in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020, pp. 1–14.
 - [32] C-track [Online]. Available: <https://www.youtube.com/watch?v=9FAdXlwPCgU>